Software Project Concept Notes

System architecture overview, requirement specifications, and execution baseline.

1. Project Name

StudyLoft
Group Name: DINC
Group Members: Delfin Kütükçü, Işıkalp Karakaş, Nildeniz Efe, Nadide Zeynep Dedeoğlu, Çınar Aktaş

2. Short Description

Problem: Students often struggle with motivation, isolation, and lack of consistency during long study sessions.
Target Users: High school, university students, and study groups.
Core Purpose & Solution: StudyLoft is a multiplayer virtual apartment study platform designed to keep students accountable. It provides a ground-floor virtual library where users can study together with real-time avatars using integrated study timers. Earning study points unlocks furniture and clothing to decorate their own floor, mixing productivity with gamification.

3. Requirements

- 1. The system must provide a real-time multiplayer ground-floor virtual library allowing users to join others at adjacent tables using text and emoji interactions only (voice/video chat is excluded to minimize distraction).
- 2. The system must include an integrated study timer that calculates active study session durations, featuring an anti-cheat mechanism that auto-pauses when a user switches browser tabs.
- 3. The system must feature a gamification engine that awards points based on completed study durations and supports a tiered reward system for unlocking progressively rarer furniture.
- 4. The platform must include a virtual store where users can redeem earned points for tiered furniture and clothing.
- 5. The system must provide a personal apartment floor decorator allowing users to place and arrange unlocked items.

- 6. The platform must offer a comprehensive productivity analytics dashboard featuring leaderboards, performance graphs, and weekly habit tracking reports.
- 7. The system must synchronize multiplayer avatar states and session data in real-time, including an automatic WebSocket connection recovery protocol for dropped sessions.
- 8. The system response time for real-time synchronization must remain under 200ms to ensure a seamless experience.
- 9. The system must securely manage user authentication and active user sessions while protecting data privacy.
- 10. The architecture must be scalable to handle concurrent connections efficiently using modern web/app tech stacks.

4. Simple Pseudocode for Implementation Parts

```
None
FUNCTION startStudySession(userID, tableID):
    IF user is logged in AND tableID is available:
        user.state = "STUDYING"
        UPDATE_AVATAR_PRESENCE(userID, tableID, "STUDYING")
        sessionTimer = START_TIMER()

        // Anti-Cheat Mechanism
        ON BROWSER_TAB_SWITCH_OR_MINIMIZE:
            PAUSE_TIMER(sessionTimer)
            UPDATE_AVATAR_PRESENCE(userID, tableID, "IDLE
(PAUSED)")

        RETURN "Session started at table " + tableID
    ELSE:
        RETURN "Error: Unable to start session"
END FUNCTION

FUNCTION calculatePointsAndReward(durationMinutes):
    IF durationMinutes >= 15:
        pointsEarned = durationMinutes * 5
```

```
        UPDATE_USER_POINTS(userID, pointsEarned)

        // Tiered Reward Logic
        unlockedTier = CALCULATE_FURNITURE_TIER(user.totalPoints)

        RETURN "You earned " + pointsEarned + " points! Current
Unlocked Tier: " + unlockedTier
    ELSE:
        RETURN "Study longer to earn points."
END FUNCTION
```

5. Sample Views

- - View 1: Ground-Floor Virtual Library View
  - Layout: Grid layout representing a library. Features tables with empty/occupied seats, online avatars displaying current status (e.g., studying), text/emoji chat bubbles, and a global study timer overlay.
- - View 2: Personal Apartment Floor Decorator View
  - Layout: Blank room grid. Features a sidebar inventory containing unlocked, tiered furniture/clothing items, and drag-and-drop customization grid to place items.
- - View 3: Analytics & Leaderboard Dashboard
  - Layout: Dashboard with multiple widget panels. Features weekly charts displaying study hours, habit tracking stats, and a global leaderboard.

6. Sample Design Information

- - Main Components: Frontend UI (renders environment, text/emoji overlays, and dashboards), Real-time WebSocket Server (handles avatar movements, timers, and connection recovery), Analytics Engine (processes study data for reports).
- - Data Entities:
  - User: {userID, username, points, avatar_outfit, furniture_tier}
  - Session: {sessionID, userID, duration, date, paused_count}
  - Item: {itemID, name, type, cost, tier_level}
  - FloorLayout: {layoutID, userID, placedItems[]}

- - Tech Stack: React (Frontend), Node.js (Backend), WebSockets/Socket.io (Real-time Sync with auto-reconnect logic), PostgreSQL (Relational Database), Gemini API (Analytics Insights).
- - Multiplayer Concept Support: WebSockets maintain persistent connections for low-latency bidirectional data flow, broadcasting avatar states and text/emoji messages to connected clients. It utilizes connection recovery with exponential backoff to re-sync avatars smoothly if the network drops.

7. Scenarios

- - Scenario 1: Earning points through studying
  - User: Delfin (Student)
  - Goal: Complete a study session and earn points.
  - Preconditions: Delfin is logged in, has zero points, and is seated in the Virtual Library.
  - Step-by-Step User Actions: 1) Delfin starts a 30-minute timer. 2) Delfin focuses on work until the timer finishes (without switching tabs, keeping the anti-cheat inactive). 3) Delfin clicks "Finish Session".
  - Postconditions: Delfin's account is credited with 150 points. A new session record is saved in the database.
- - Scenario 2: Decorating the apartment
  - User: Çınar (Student)
  - Goal: Buy and place a new desk in his apartment.
  - Preconditions: Çınar has 500 points and is in the Virtual Store.
  - Step-by-Step User Actions: 1) Çınar purchases a tier-2 desk for 300 points. 2) He navigates to his Personal Apartment View. 3) He drags the desk from his inventory and places it on the customization grid. 4) He clicks "Save Layout".
  - Postconditions: Çınar's points decrease to 200. The FloorLayout entity is updated to include the desk coordinates. The UI reflects the new room layout.